

LostPatrol

Physical AI 분실물 탐색·순찰 로봇 — 현재 진행상황 보고서

작성일: 2026.08.25 | 팀원: 백경률 · 이윤정 · 조은수 | Live Web: <https://BaekGyeongRyul.github.io/LostPatrol/>

현재 한 줄 상태: Web-Supabase-Robot Controller 및 Safety Monitor의 기본 데이터 연동과 Vision 3종 객체 분류 모델 구축까지 완료되었으며, 현재 실제 Razbot 하드웨어·카메라·YOLO·자동순찰을 하나의 시스템으로 통합하는 단계에 있습니다.

1. 프로젝트 개요

LostPatrol은 공공장소를 순찰하는 로봇이 분실물 후보(가방·우산·물병)를 탐지하고, 탐지 정보를 Supabase에 저장하여 관리자 웹사이트에서 조회·관리할 수 있도록 하는 Physical AI 기반 프로젝트입니다. Web과 Raspberry Pi/Razbot, Vision 파이프라인은 서로 직접 통신하지 않고 Supabase를 중앙 데이터 허브로 사용합니다.

제어/상태: Web ↔ Supabase(5개 테이블 + Storage) ↔ Raspberry Pi / Razbot
탐지: Razbot Camera → OpenCV / YOLO → Supabase → Web

2. 담당 역할

백경률	이윤정	조은수
Web · Supabase · Git/GitHub · System Integration	Raspberry Pi/Razbot · Robot Controller · Safety Monitor · Line Tracking/Ultrasonic/Camera	Python · OpenCV · Roboflow · YOLO/Vision

3. 주요 진행 상황 — 백경률 (Web / Supabase / GitHub)

- ✅ React + Vite 관리자 웹사이트 구축 (Dashboard · Robot Control · 실시간 순찰 · Lost Items)
- ✅ 감사님·팀원 피드백 반영 웹 리디자인 — 현대자동차 공식 웹사이트의 **기업형 레이아웃(넓은 화면, 밝고 정돈된 톤, 상단 내비게이션)**을 디자인 모티브로 참고(로그·고유 디자인 복제 아님), 실제 서비스형 UI로 개선
- ✅ Footer에 팀원 이름 추가 —
COPYRIGHT © 2026 LostPatrol Team · 백경률 · 이윤정 · 조은수 ALL RIGHTS RESERVED
- ✅ Robot Control UI 개선 — 불필요 영역 축소, 자동차 모티브 컨트롤러를 화면 중심에 배치, 키보드+마우스 클릭+모바일 터치 조작 지원
- ✅ 수동/자동순찰 명령 Web → Supabase 실제 연동 확인 —
forward · backward · left(제자리 좌회전) · right(제자리 우회전) · stop · capture · patrol_start · patrol_stop
- ✅ Supabase 구성 — **robot_commands, robot_status, safety_status, patrol_events, lost_items** + Storage **lost-item-photos**, 담당자별 anon 권한/RLS 설정 및 실제 연동 테스트 완료
- ✅ Git/GitHub 팀 협업 구조 구축 — 공용 Repository, Data Contract 기반 담당자별 개발 가이드
- ✅ GitHub Pages 실제 배포 — Live Web: <https://BaekGyeongRyul.github.io/LostPatrol/>

4. 팀원별 진행상황

이윤정 — Raspberry Pi / Razbot

- ✅ Mock Robot Controller, command polling, heartbeat
- ✅ 순찰 상태머신, 안전정지 원칙
- ✅ Supabase 연동 — forward/patrol_start/patrol_stop 왕복, Web ONLINE 표시 확인
- ✅ Storage 이미지 업로드 실제 확인
- ✅ safety_status 연동, patrol_events 이벤트 로그
- ✅ safety_monitor.py ↔ Supabase 기본 연동 검증
- ✅ 개발 중 버그 수정 — 한글경로 cv2.imwrite, upsert 권한, Thread 예외 전달

※ 실제 Razbot 하드웨어(모터/센서/카메라) 전체 통합은 아직 진행 중입니다.

조은수 — Vision / YOLO

- ✅ YOLO + OpenCV 방식 결정, Gemini API 미사용 결정
- ✅ Roboflow 활용, Bounding Box 라벨링
- ✅ 가방/우산/물병 3종으로 탐지 범위 확정
- ✅ 3종 객체를 정상적으로 구별하는 모델 구축 완료

※ Razbot 카메라 실제 연동, Raspberry Pi 추론, Supabase 자동등록 등 Vision 최종 통합은 진행 중입니다.

5. 감사 피드백 반영

- 웹사이트를 실제 서비스처럼 보이도록 개선
- Footer에 팀원 이름 표시
- Robot Control 컨트롤러 중심 UI로 변경
- 분실물 종류를 가방/우산/물병 3종으로 제한
- Gemini API 대신 OpenCV+YOLO 직접 학습
- Roboflow Bounding Box 라벨링 채택
- Git 기반 팀 협업 방식 적용
- 자동순찰은 Line Tracking 방식 채택
- Ultrasonic 센서 기반 안전정지 방향 확정

6. 현재 시스템 연동 상태

완료	진행 중
Web ↔ Supabase / Robot Controller ↔ Supabase Heartbeat ↔ Web ONLINE/OFFLINE Storage 이미지 업로드 Safety Monitor ↔ Supabase Vision 3종 모델 구축 GitHub Collaboration / GitHub Pages	실제 Razbot Hardware / Motor Control Line Tracking / Ultrasonic Safety Stop Razbot Camera 실제 연동 Raspberry Pi YOLO inference Vision → lost_items 자동등록 Robot + Vision + Web 최종 통합

7. 남은 핵심 과제 (우선순위 순)

- 실제 Razbot 하드웨어 및 모터 제어 통합
- Line Tracking 자동순찰 실제 동작 검증
- Ultrasonic 장애물 안전정지 실제 검증
- Razbot Camera → YOLO inference 연결
- Vision 탐지 결과 → lost_items / Storage 자동등록
- Web에서 실제 탐지 결과 자동 표시
- 전체 시스템 통합 테스트 및 최종 시연 리허설